REVISION CHEAT SHEET:

Exception Handling in C#

TOPIC 18

Interview-ready notes · examples · C# · diagrams

Topic 18 of 40

Applications fail.

Files may be missing.

Databases may become unavailable.

An HTTP service may time out.

A caller may provide an invalid argument.

A collection lookup may fail.

A business operation may reach an invalid state.

C# represents many abnormal failures using:

EXAMPLE / FLOW

```
Exceptions
```

Exception handling gives your application a structured way to:

EXAMPLE / FLOW

```
Detect failure

Stop normal execution

Propagate failure information

Handle recoverable failures

Clean up resources

Log diagnostic information

Translate failures at architectural boundaries
```

The central mental model is:

EXAMPLE / FLOW

```
Normal Path
    ↓
Something Fails
    ↓
Exception Thrown
    ↓
Runtime Searches Call Stack
    ↓
Matching catch?
    ├─ Yes → Handle
    └─ No  → Continue Propagating
```

The core C# constructs are:

C#

```
try

catch

finally

throw
```

TOPIC 18

C# supports `try-catch`, `try-finally`, and `try-catch-finally`. When an exception is thrown, the runtime searches for a compatible handler; only one matching `catch` handles that thrown exception.

Why Exception Handling Exists

Imagine:

C#

```
Gift gift = repository.GetGift( 1001);

ProcessGift(
    gift);
```

The repository operation might fail because:

EXAMPLE / FLOW

```
Database unavailable

Network interrupted

Connection timeout

Malformed data

Unexpected provider failure
```

Without a structured error mechanism, every method might need:

EXAMPLE / FLOW

```
Return code 0

Return code -1

Return code -27

Check global error state

Check null

Check another flag
```

Exceptions provide a standardized mechanism for propagating abnormal failure information through multiple method layers.

Exception Flow

Suppose:

EXAMPLE / FLOW

```
Controller
    ↓
```

```
GiftService
    ↓
GiftRepository
    ↓
Database
```

A repository operation throws:

EXAMPLE / FLOW

```
TimeoutException
```

If repository does not catch it:

EXAMPLE / FLOW

```
Database
    ↑
GiftRepository
    ↑
GiftService
    ↑
Controller / Global Handler
```

The exception **propagates upward through the call stack** until:

EXAMPLE / FLOW

```
A compatible catch handles it
```

or:

EXAMPLE / FLOW

```
It remains unhandled
```

What Is an Exception?

An exception is an object derived from:

C#

```
System.Exception
```

It contains information describing a failure.

Common properties include:

EXAMPLE / FLOW

```
Message

StackTrace

InnerException

Source

Data
```

Example:

C#

```
try
{
    int result = int.Parse( "ABC");
```

```
}
catch (FormatException ex)
{
    Console.WriteLine(
        ex.Message);
}
```

Here:

EXAMPLE / FLOW

```
FormatException
```

is the concrete exception object.

Exception Hierarchy

Conceptually:

EXAMPLE / FLOW

```
System.Object
    ↓
System.Exception
    ↓
Specific exception types
```

Examples:

EXAMPLE / FLOW

```
Exception
    |
    +-- ArgumentException
    |       |
    |       +-- ArgumentNullException
    |       |
    |       +-- ArgumentOutOfRangeException
    |
    +-- InvalidOperationException
    |
    +-- FormatException
    |
    +-- IOException
    |
    +-- TimeoutException
    |
    +-- OperationCanceledException
```

Your own custom exception also derives from:

C#

```
Exception
```

when no existing framework exception accurately communicates the failure.

The `throw` Statement

Definition

`throw` raises an exception.

Example:

```C#
if (amount <= 0m)
{
    throw new ArgumentOutOfRangeException(
        nameof(amount),
        "Gift amount must be positive.");
}
```

Execution stops at:

```C#
throw
```

and the runtime begins searching for a compatible handler.

The expression supplied to `throw` must produce something convertible to `System.Exception`.

Basic Throw Flow

```EXAMPLE / FLOW
Validate Amount
    ↓
Amount > 0?
    ├─ Yes → Continue
    └─ No
         ↓
       throw
         ↓
   Current Method Stops
         ↓
   Search for catch
```

Built-In Exceptions First

Before creating a custom exception, ask:

```EXAMPLE / FLOW
Does .NET already have
an appropriate exception type?
```

For example:

```C#
ArgumentNullException

ArgumentException

ArgumentOutOfRangeException

InvalidOperationException
```

Microsoft recommends using predefined exception types when one accurately represents the error.

ArgumentNullException

Use when a required argument is unexpectedly null.

Example:

```C#
public void ProcessGift(
    Gift gift)
{
    if (gift is null)
    {
        throw new ArgumentNullException(
            nameof(gift));
    }
}
```

Modern helper:

```C#
ArgumentNullException.ThrowIfNull(
    gift);
```

This is clearer and concise.

ArgumentException

Use when an argument is invalid but a more specific argument exception doesn't fit.

Example:

```C#
if (
    currencyCode.Length
    !=
    3)
{
    throw new ArgumentException(
        "Currency code must contain three characters.",
        nameof(currencyCode));
}
```

ArgumentOutOfRangeException

Use when the argument's type is acceptable but its value is outside the valid range.

```C#
if (amount <= 0m)
{
    throw new ArgumentOutOfRangeException(
        nameof(amount),
        amount,
        "Amount must be positive.");
}
```

InvalidOperationException

Use when:

```EXAMPLE / FLOW
The method itself is valid

but
```

```
the object's current state
does not permit the operation
```

TOPIC 18

Example:

```C#
public void Complete()
{
    if (
        Status
        ==
        GiftStatus.Cancelled)
    {
        throw new InvalidOperationException(
            "A cancelled gift cannot be completed.");
    }

    Status = GiftStatus.Completed;
}
```

The problem is not:

```
EXAMPLE / FLOW
Invalid argument
```

It is:

```
EXAMPLE / FLOW
Invalid object state
```

Microsoft specifically recommends `InvalidOperationException` when a method call is inappropriate for the object's current state.

FormatException

Common parsing example:

```C#
int value = int.Parse( "ABC");
```

This throws:

```
EXAMPLE / FLOW
FormatException
```

because the input cannot be interpreted as the requested format.

For routine parsing where invalid input is expected:

```C#
int.TryParse(...)
```

is usually preferable.

KeyNotFoundException

Example:

```C#
Dictionary<long, Gift>
    gifts = [];

Gift gift = gifts[1001];
```

If the key isn't present:

```EXAMPLE / FLOW
KeyNotFoundException
```

may be thrown.

If missing keys are expected:

```C#
gifts.TryGetValue(
    1001,
    out Gift? gift);
```

is often better.

TimeoutException

Represents a timeout condition when the API being used models timeout through this exception type.

Example conceptual flow:

```EXAMPLE / FLOW
Call external system
    ↓
Time limit exceeded
    ↓
TimeoutException
```

Whether to retry depends on:

```EXAMPLE / FLOW
Operation type

Idempotency

Provider semantics

Retry policy

Business rules
```

Do not blindly retry every timeout.

IOException

Used for many I/O-related failures.

Examples may involve:

```EXAMPLE / FLOW
File operations

Streams
```

```
Device operations
```


Catch a more specific exception if you have meaningful handling for it.

OperationCanceledException

Cancellation deserves special treatment.

Many async APIs use:

EXAMPLE / FLOW

```
OperationCanceledException
```

when a cancellation token is observed.

Microsoft recommends generally catching `OperationCanceledException` rather than its more specific `TaskCanceledException` when handling cancellation from asynchronous APIs.

Cancellation Is Usually Not an Application Failure

Conceptually:

EXAMPLE / FLOW

```
Failure:
"Something unexpectedly broke."
```

versus:

EXAMPLE / FLOW

```
Cancellation:
"The operation was intentionally asked to stop."
```

Do not routinely log cancellation as:

EXAMPLE / FLOW

```
Critical system failure
```

unless your context actually requires that interpretation.

The try Block

Code that may throw can be placed inside:

C#

```
try
{
}
```

Example:

C#

```
try
{
    ProcessGift(
        gift);
}
```

The `try` block by itself is incomplete.

It must be followed by:

EXAMPLE / FLOW

```
catch

finally

or both
```

try-catch

Basic form:

C#

```
try
{
    ProcessGift(
        gift);
}
catch (InvalidOperationException ex)
{
    Console.WriteLine(
        ex.Message);
}
```

Flow:

EXAMPLE / FLOW

```
Enter try
    ↓
ProcessGift()
    ↓
Exception?
    ├─ No → Skip catch
    └─ Yes
          ↓
Matching catch?
          ↓
Handle
```

No Exception

C#

```
try
{
    Console.WriteLine(
        "Processing");
}
catch (Exception)
{
    Console.WriteLine(
        "Failure");
}
```

If no exception occurs:

EXAMPLE / FLOW

```
try executes

catch does not
```

TOPIC 18

Multiple catch Blocks

You can handle different exception types separately.

C#

```
try
{
    await ProcessGiftAsync(
        gift,
        cancellationToken);
}
catch (
    ArgumentException ex)
{
    Console.WriteLine(
        $"Invalid input: {ex.Message}");
}
catch (
    OperationCanceledException)
{
    Console.WriteLine(
        "Gift processing was cancelled.");
}
catch (
    TimeoutException ex)
{
    Console.WriteLine(
        $"Timed out: {ex.Message}");
}
```

The runtime checks matching catch clauses in order.

Catch Specific Exceptions Before General Ones

Because:

EXAMPLE / FLOW

```
ArgumentNullException
IS AN
ArgumentException
```

this order makes sense:

C#

```
try
{
}
catch (
    ArgumentNullException ex)
{
}
catch (
    ArgumentException ex)
{
}
catch (
```

```
    Exception ex)
{
}
```


Think:

EXAMPLE / FLOW

```
Most specific
    ↓
More general
    ↓
Exception
```

Microsoft recommends ordering handlers from the most-derived to the least-derived exception types.

Invalid Catch Ordering

Conceptually:

C#

```
try
{
}
catch (
    Exception ex)
{
}
catch (
    InvalidOperationException ex)
{
}
```

The second handler can never be reached because:

EXAMPLE / FLOW

```
InvalidOperationException
IS AN
Exception
```

and the earlier handler already catches it.

Catch Without Exception Variable

If you don't need the object:

C#

```
catch (
    OperationCanceledException)
{
    Console.WriteLine(
        "Cancelled.");
}
```

No need to declare:

C#

```
ex
```

if you won't use it.

Catch-All

You can write:

```C#
catch
{
}
```

It catches exceptions without declaring a type variable.

If used, it must be the final general handler.

In normal application code, a typed:

```C#
catch (Exception ex)
```

at a true boundary is often more useful because you can log diagnostics.

Never Swallow Exceptions Accidentally

Bad:

```C#
try
{
    await repository.SaveAsync(
        gift,
        cancellationToken);
}
catch (Exception)
{
}
```

Now:

EXAMPLE / FLOW

```
Database failed

Operation didn't complete

Caller believes nothing went wrong

Diagnostic information disappears
```

This is extremely dangerous.

Better Options

If you cannot meaningfully recover:

EXAMPLE / FLOW

```
Don't catch
```

or:

EXAMPLE / FLOW

```
Log and rethrow
```

or:

EXAMPLE / FLOW

```
Translate at an architectural boundary
```

Microsoft's guidance is clear: if your code cannot recover from an exception, don't catch it merely to suppress it; allow an appropriate higher layer to handle it.

What Does "Handle" Actually Mean?

Handling does **not** mean:

EXAMPLE / FLOW

```
I wrote a catch block.
```

A catch genuinely handles an exception when it can do something meaningful such as:

EXAMPLE / FLOW

```
Recover safely

Use an alternate source

Return a meaningful result

Translate to a domain/application error

Retry when safe

Restore state

Produce a correct boundary response
```

Bad Catch

C#

```
catch (Exception ex)
{
    Console.WriteLine(
        ex.Message);
}
```

and then continue as though the operation succeeded.

You may have created a more dangerous state than allowing the request to fail.

Exception Propagation

Suppose:

C#

```
static void A()
{
    B();
}

static void B()
{
    C();
}
```

```
static void C()
{
    throw new InvalidOperationException(
        "Processing failed.");
}
```

Call:

C#
```
A();
```

Flow:

EXAMPLE / FLOW
```
A()
 ↓
B()
 ↓
C()
 ↓
throw
 ↑
No catch in C
 ↑
No catch in B
 ↑
No catch in A
 ↑
Caller
```

This process is often called:

EXAMPLE / FLOW
```
Stack unwinding
```

when frames are exited while locating a handler.

Stack Trace

The exception's stack trace shows the call path leading toward the failure.

Conceptually:

EXAMPLE / FLOW
```
GiftRepository.SaveAsync
    ↓
GiftService.CreateAsync
    ↓
GiftsController.Create
```

This information is extremely valuable for debugging.

That is why preserving stack traces when rethrowing is important.

Rethrowing an Exception

Suppose you want to:

EXAMPLE / FLOW

```
Log
+
let higher layer still handle
```

Correct:

C#

```
try
{
    ProcessGift();
}
catch (Exception ex)
{
    Log(
        ex);

    throw;
}
```

The bare:

C#

```
throw;
```

preserves the existing stack trace.

Wrong Rethrow

Avoid:

C#

```
catch (Exception ex)
{
    Log(
        ex);

    throw ex;
}
```

Why?

Because:

C#

```
throw ex;
```

changes/restarts stack-trace information from the current rethrow point, hiding useful information about the original origin. Microsoft recommends bare `throw;` inside the current handler.

Correct Mental Model

EXAMPLE / FLOW

```
throw;
    →
Rethrow current exception
and preserve original trace
```

EXAMPLE / FLOW

```
throw ex;
    →
Throw this expression again
and damage original trace context
```


Rethrowing Outside the catch Block

Sometimes an exception needs to be captured and rethrown later.

Microsoft recommends:

```
C#
ExceptionDispatchInfo
```

for that situation.

Conceptually:

```
C#
ExceptionDispatchInfo
    .Capture(
        ex)
    .Throw();
```

This preserves exception dispatch information when rethrowing outside the original handler.

This is an advanced case.

Most normal business code should simply use:

```
C#
throw;
```

inside the `catch`.

Wrapping an Exception

Sometimes a lower-level failure should be translated into a higher-level exception while preserving the original exception.

Example:

```
C#
try
{
    await externalClient.SendAsync(
        request,
        cancellationToken);
}
catch (
    HttpRequestException ex)
{
    throw new GiftReceiptDeliveryException(
        "The gift receipt could not be delivered.",
        ex);
}
```

The original exception becomes:

Why InnerException Matters

Now callers can see:

EXAMPLE / FLOW

```
High-Level Meaning
    →
Gift receipt delivery failed
```

and still inspect:

EXAMPLE / FLOW

```
Low-Level Cause
    →
HttpRequestException
```

Flow:

EXAMPLE / FLOW

```
HttpRequestException
        ↓
wrapped by
        ↓
GiftReceiptDeliveryException
        ↓
InnerException
retains original cause
```

Do Not Destroy the Original Cause

Bad:

C#

```
catch (
    HttpRequestException)
{
    throw new GiftReceiptDeliveryException(
        "Delivery failed.");
}
```

You have lost the original exception object.

Better:

C#

```
catch (
    HttpRequestException ex)
{
    throw new GiftReceiptDeliveryException(
        "Delivery failed.",
        ex);
}
```

Exception Translation

Exception translation can be useful at architectural boundaries.

Example:

```
EXAMPLE / FLOW
Infrastructure exception
    ↓
Application/domain exception
```

Possible flow:

```
EXAMPLE / FLOW
HttpRequestException
    ↓
PaymentGatewayException
```

or:

```
EXAMPLE / FLOW
Provider-specific database failure
    ↓
Repository/application exception
```

But don't translate every exception blindly.

Translate when the higher-level exception provides:

```
EXAMPLE / FLOW
Meaningful abstraction

Stable contract

Useful context
```

Do Not Build Exception Lasagna

Bad:

```
EXAMPLE / FLOW
SqlException
    ↓
RepositoryException
    ↓
ServiceException
    ↓
ControllerException
    ↓
ApiException
```

where every layer merely wraps without adding useful meaning.

This creates noise and deeply nested `InnerException` chains.

Add Context When Wrapping

Good:

```C#
throw new GiftReceiptDeliveryException(
    $"Receipt delivery failed for Gift {giftId}.",
    ex);
```

Now diagnostic context includes:

EXAMPLE / FLOW

```
Operation

Gift identifier

Original cause
```

Do not include secrets or sensitive information.

finally

Definition

A `finally` block contains code that should execute when control leaves the corresponding `try`.

Example:

```C#
bool processing = false;

try
{
    processing = true;

    ProcessGift();
}
finally
{
    processing = false;
}
```

`finally` executes when leaving `try` through normal completion, exception propagation, or flow statements such as `return`, `break`, or `continue`. There are extreme process-termination cases where it may not execute, so it should not be interpreted as an absolute guarantee under every possible runtime termination.

try-finally

You do not need a `catch` if your goal is cleanup.

```C#
try
{
    UseTemporaryState();
}
finally
{
    RestoreState();
}
```

The exception still propagates after `finally` unless handled elsewhere.

try-catch-finally

TOPIC 18

```C#
try
{
    ProcessGift();
}
catch (
    InvalidOperationException ex)
{
    Log(
        ex);
}
finally
{
    Cleanup();
}
```

Flow:

EXAMPLE / FLOW

```
try
    ↓
Exception?
    ↓
catch
    ↓
finally
    ↓
Continue
```

finally Is for Cleanup, Not Error Suppression

Bad:

```C#
finally
{
    throw new Exception(
        "Cleanup failed.");
}
```

Throwing from `finally` can mask the original exception.

Microsoft recommends avoiding exceptions from `finally` blocks.

Resource Cleanup

Historically:

```C#
Stream? stream = null;

try
{
    stream = File.OpenRead( path);

    // Use stream.
}
finally
```

```
{
    stream?.Dispose();
}
```


Modern C# usually prefers:

```C#
using Stream stream = File.OpenRead( path);
```

when the resource implements `IDisposable`.

The `using` statement is compiled into cleanup behavior based on `try-finally`, ensuring disposal when control leaves the scope.

`IDisposable` and deterministic resource management are Topic 19.

Exception Filters

C# supports:

```C#
when
```

on `catch`.

Example:

```C#
try
{
    ProcessGift(
        gift);
}
catch (
    InvalidOperationException ex)
    when (
        gift.Status
        ==
        GiftStatus.Cancelled)
{
    Console.WriteLine(
        "Cancelled gifts cannot be processed.");
}
```

The handler runs only when:

EXAMPLE / FLOW

```
Exception type matches
+
filter returns true
```

Multiple Filters for Same Type

```C#
try
{
    Process(
        gift);
}
catch (
```

```
    InvalidOperationException ex)
    when (
        gift.Status
        ==
        GiftStatus.Cancelled)
{
    HandleCancelledGift(
        ex);
}
catch (
    InvalidOperationException ex)
    when (
        gift.Status
        ==
        GiftStatus.Refunded)
{
    HandleRefundedGift(
        ex);
}
catch (
    InvalidOperationException ex)
{
    HandleOtherStateFailure(
        ex);
}
```

Filters can distinguish multiple handlers for the same exception type.

Why Filters Are Better Than catch + if + throw

Alternative:

C#

```
catch (
    InvalidOperationException ex)
{
    if (
        ShouldHandle(
            ex))
    {
        Handle(
            ex);
    }
    else
    {
        throw;
    }
}
```

Filter:

C#

```
catch (
    InvalidOperationException ex)
    when (
        ShouldHandle(
            ex))
{
    Handle(
        ex);
}
```

The filter expresses intent more directly.

TOPIC 18

Exception Filters and Stack Unwinding

Exception filters are evaluated **before the stack is unwound for that candidate handler**.

That means if a filter returns false, the search continues while preserving more original failure context for debugging. This is one of the important technical advantages of `when` filters.

Do Not Put Dangerous Side Effects in Filters

A filter should generally answer:

EXAMPLE / FLOW

```
Should this handler handle
this exception?
```

Avoid using filters for important state-changing operations.

Prefer:

C#

```
when (
    ex.ErrorCode
    ==
    knownCode)
```

over:

C#

```
when (
    ChargeCustomer()
    &&
    SendEmail())
```

Throw Expressions

`throw` can also appear in certain expressions.

Example:

C#

```
Gift gift = input ?? throw new ArgumentNullException( nameof(input));
```

Useful for constructor assignment:

C#

```
public GiftService(
    IGiftRepository repository)
{
    _repository = repository ?? throw new ArgumentNullException( nameof(repository));
}
```

C# supports throw expressions in contexts such as null-coalescing expressions, conditional expressions, and expression-bodied members.

Prefer ThrowIfNull for Simple Guards

Instead of:

```C#
_repository = repository ?? throw new ArgumentNullException( nameof(repository));
```

modern .NET code can often simply use:

```C#
ArgumentNullException.ThrowIfNull(
    repository);

_repository = repository;
```

Either can be reasonable.

Exceptions Are Not Normal Control Flow

Bad:

```C#
try
{
    int value = int.Parse( input);
}
catch (
    FormatException)
{
    // Expected every time invalid user input occurs.
}
```

If invalid input is common and expected:

```C#
if (
    int.TryParse(
        input,
        out int value))
{
}
```

is usually better.

Microsoft explicitly recommends avoiding exceptions for routine conditions and using `Try*` APIs when appropriate.

Exceptional vs Expected

Think:

EXAMPLE / FLOW

```
Expected routine branch
    →
if / switch / Try* / result
```

EXAMPLE / FLOW

```
Abnormal failure
    →
Exception
```

Example — Gift Lookup

If a missing gift is a normal possibility:

```C#
Gift? gift = await repository.FindAsync( giftId);
```

then:

```C#
if (gift is null)
{
    return NotFound();
}
```

may be better than:

```C#
try
{
    ...
}
catch (
    GiftNotFoundException)
{
}
```

for every ordinary lookup miss.

But Custom NotFound Exceptions Can Still Be Useful

Some architectures intentionally use:

```EXAMPLE / FLOW
Application command/query
    ↓
Throw NotFoundException
    ↓
Global exception handler
    ↓
HTTP 404
```

This can be valid when consistently designed.

The important rule is:

```EXAMPLE / FLOW
Don't use exceptions casually
for hot-path routine branching
without architectural reason.
```

Custom Exceptions

When built-in exceptions do not communicate the problem accurately, create your own.

Example:

```C#
public sealed class GiftProcessingException :
    Exception
```

```
{
    public GiftProcessingException()
    {
    }

    public GiftProcessingException(
        string message)
        :
        base(
            message)
    {
    }

    public GiftProcessingException(
        string message,
        Exception innerException)
        :
        base(
            message,
            innerException)
    {
    }
}
```

Microsoft's current guidance recommends ending custom exception names with `Exception`, deriving from `Exception`, and providing the common parameterless, message, and message-plus-inner-exception constructors.

Custom Exception Naming

Good:

EXAMPLE / FLOW

```
GiftProcessingException

PaymentGatewayException

GiftAllocationException
```

Bad:

EXAMPLE / FLOW

```
GiftProblem

BadGiftError

SomethingWentWrong
```

The suffix:

EXAMPLE / FLOW

```
Exception
```

makes the type's purpose immediately clear.

Add Useful Properties When Needed

Example:

```C#
public sealed class GiftAllocationException :
    Exception
{
    public GiftAllocationException(
        long giftId,
        decimal attemptedAmount,
        string message)
        :
        base(
            message)
    {
        GiftId = giftId;

        AttemptedAmount = attemptedAmount;
    }

    public long GiftId
    {
        get;
    }

    public decimal AttemptedAmount
    {
        get;
    }
}
```

Add properties only when callers or diagnostic systems can meaningfully use them. Microsoft recommends additional custom exception properties only when they provide programmatic value.

Do Not Store Sensitive Information in Exceptions

Avoid:

```EXAMPLE / FLOW
Passwords

Tokens

Connection strings

Full payment details

Secrets
```

in:

```EXAMPLE / FLOW
Message

Data

Custom properties
```

Exceptions frequently reach:

```EXAMPLE / FLOW
Logs

Telemetry
```

```
Monitoring

Crash reports
```

TOPIC 18

Exception Messages

Good:

EXAMPLE / FLOW

```
"Gift amount must be positive."
```

Bad:

EXAMPLE / FLOW

```
"Error."
```

Also bad:

EXAMPLE / FLOW

```
"Something went wrong somewhere maybe."
```

Messages should provide useful diagnostic information without exposing sensitive data.

Message vs Type

Code should generally react to:

EXAMPLE / FLOW

```
Exception type

Structured exception properties
```

rather than parsing:

EXAMPLE / FLOW

```
Message text
```

Bad:

C#

```
if (
    ex.Message.Contains(
        "timeout"))
{
}
```

Messages may change or be localized.

Prefer:

C#

```
catch (
    TimeoutException)
```

or a structured property/code where available.

InnerException

Example:

```
C#
catch (
    HttpRequestException ex)
{
    throw new GiftReceiptDeliveryException(
        $"Receipt delivery failed for Gift {giftId}.",
        ex);
}
```

Then:

```
C#
ex.InnerException
```

on the custom exception identifies the underlying cause.

Exception Data

`Exception.Data` can carry additional key/value diagnostic data.

Example:

```
C#
ex.Data["GiftId"] = giftId;
```

But custom typed properties are often easier to consume when information is an important part of the exception contract.

Never put sensitive values in either location.

Catch Only What You Can Handle

Bad low-level code:

```
C#
try
{
    await repository.SaveAsync(
        gift,
        cancellationToken);
}
catch (Exception)
{
    return false;
}
```

Potential problems:

EXAMPLE / FLOW

```
Cancellation becomes false

Out-of-memory style failures become false

Programming bugs become false
```

```
Database outage becomes false
```


Everything loses meaning.

Better

Catch specific failures you actually understand.

```C#
try
{
    await sender.SendAsync(
        gift,
        cancellationToken);
}
catch (
    TimeoutException ex)
{
    // Known policy for timeout.
}
```

Let unknown failures propagate to a higher-level boundary.

Catch Exception at True Boundaries

Catching:

```C#
Exception
```

can be appropriate at boundaries such as:

EXAMPLE / FLOW

```
ASP.NET Core exception middleware

Background worker loop boundary

Message-consumer boundary

Top-level application boundary
```

because the boundary needs to:

EXAMPLE / FLOW

```
Log

Return standardized failure response

Prevent one request/message
from crashing unrelated work
```

Logging Exceptions

With `ILogger`:

```C#
catch (
    Exception ex)
```

```
{
    _logger.LogError(
        ex,
        "Gift processing failed for Gift {GiftId}.",
        giftId);

    throw;
}
```

Passing the exception object preserves structured diagnostics.

Do Not Log the Same Exception at Every Layer

Bad:

EXAMPLE / FLOW

```
Repository logs

Service logs same exception

Controller logs same exception

Middleware logs same exception
```

Result:

EXAMPLE / FLOW

```
Four log entries
for one failure
```

This creates noise.

Choose a clear logging boundary.

Log + Rethrow Only When Layer Adds Value

A service may log only if it adds useful context:

EXAMPLE / FLOW

```
GiftId

CampaignId

Operation name

Correlation ID
```

Otherwise let the central boundary log it.

Async Exceptions

Consider:

C#

```
async Task ProcessAsync()
{
    await SomeOperationAsync();

    throw new InvalidOperationException(
```

```
        "Processing failed.");
}
```

Calling:

```C#
Task task = ProcessAsync();
```

produces a task.

When asynchronous code faults, the exception is stored in the task and is observed/rethrown when the task is awaited.

Catch Async Exception

```C#
try
{
    await ProcessAsync();
}
catch (
    InvalidOperationException ex)
{
    Console.WriteLine(
        ex.Message);
}
```

The:

```EXAMPLE / FLOW
await
```

is where the asynchronous failure becomes observable to this caller.

Missing await Problem

Bad:

```C#
try
{
    ProcessAsync();
}
catch (
    Exception ex)
{
    Console.WriteLine(
        ex.Message);
}
```

The asynchronous operation isn't awaited.

The surrounding `try-catch` may finish before the asynchronous failure occurs.

Also:

```EXAMPLE / FLOW
compiler warning
```

typically alerts you that the returned task is ignored.

Better

```C#
try
{
    await ProcessAsync();
}
catch (
    Exception ex)
{
    // Handle.
}
```

Task Faulting

Conceptually:

```EXAMPLE / FLOW
Async operation starts
    ↓
Failure occurs
    ↓
Task becomes Faulted
    ↓
await task
    ↓
Exception observed
```

AggregateException Awareness

A faulted `Task` exposes:

```C#
Task.Exception
```

as an:

```EXAMPLE / FLOW
AggregateException
```

because task operations can represent multiple failures.

However, normal `await` syntax generally surfaces the relevant exception directly rather than forcing everyday code to manually inspect `AggregateException`.

Cancellation in Async Methods

```C#
try
{
    await repository.SaveAsync(
        gift,
        cancellationToken);
}
catch (
    OperationCanceledException)
    when (
        cancellationToken.IsCancellationRequested)
```

```
{
    // Expected cancellation path.
}
```


The filter can help distinguish expected caller cancellation from other cancellation contexts.

Do Not Convert Cancellation into Success

Bad:

C#

```
catch (
    OperationCanceledException)
{
    return true;
}
```

Now a cancelled operation appears successful.

Typically:

EXAMPLE / FLOW

```
Propagate cancellation
```

or map it at a well-defined boundary.

finally with async

You can use:

C#

```
try
{
    await ProcessAsync();
}
finally
{
    CleanupState();
}
```

The `finally` executes after the awaited operation completes or faults/cancels and control exits the `try`.

Exception Safety and Object State

Consider:

C#

```
gift.Status = GiftStatus.Completed;

await repository.SaveAsync(
    gift);
```

Suppose save fails.

Now the in-memory object says:

EXAMPLE / FLOW

```
Completed
```

but persistence failed.

Exception handling is also about maintaining valid state.

Prefer Operations That Preserve Invariants

Possible design:

EXAMPLE / FLOW

```
Validate

Perform required external work

Commit state when safe
```

or use a database transaction when atomic persistence is required.

The precise solution depends on your architecture.

Strong Exception Safety Concept

A useful design goal is:

> ⓘ If an operation fails, leave the object/system in a valid and understandable state.

This can involve:

EXAMPLE / FLOW

```
Transactions

Rollback

Immutable intermediate values

Compensating operations

Delayed mutation
```

Do Not Catch Programming Bugs Just to Continue

Examples:

EXAMPLE / FLOW

```
NullReferenceException

IndexOutOfRangeException

InvalidCastException
```

may indicate programming defects.

Do not routinely write:

C#

```
catch (
    NullReferenceException)
{
}
```

to hide them.

Fix the underlying bug.

Microsoft advises against explicitly throwing several runtime-reserved exceptions such as `NullReferenceException`, `IndexOutOfRangeException`, and `StackOverflowException`; use appropriate public exception types instead.

TOPIC 18

Don't Throw NullReferenceException Manually

Bad:

```C#
if (gift is null)
{
    throw new NullReferenceException();
}
```

Better:

```C#
ArgumentNullException.ThrowIfNull(
    gift);
```

because the problem is:

```EXAMPLE / FLOW
Invalid argument
```

not:

```EXAMPLE / FLOW
Runtime dereference failure
```

Exceptions and Validation

Input validation can use exceptions for programmer/API misuse.

Example:

```C#
public Gift(
    decimal amount)
{
    if (amount <= 0m)
    {
        throw new ArgumentOutOfRangeException(
            nameof(amount));
    }
}
```

But application-level user validation may instead return:

```EXAMPLE / FLOW
Validation errors

ProblemDetails

Result object

ModelState
```

depending on the layer.

TOPIC 18

Exceptions vs Result Pattern

Sometimes failures are expected business outcomes.

Example:

EXAMPLE / FLOW

```
Campaign closed

Gift already refunded

Insufficient balance

Validation failed
```

You might model them as:

C#

```
Result<T>
```

instead of exceptions.

Example concept:

C#

```
Result<Gift> result = Gift.Create(...);
```

This is a design choice.

Rule of Thumb

Use exceptions for:

EXAMPLE / FLOW

```
Unexpected or exceptional failure

Broken assumptions

Invalid operation state

Infrastructure failure

Impossible continuation
```

Use explicit result/branching for:

EXAMPLE / FLOW

```
Common expected outcomes

Validation feedback

Routine absence

Normal business alternatives
```

No single rule fits every architecture.

Exception Filters vs Business Branches

TOPIC 18

This:

```C#
catch (
    GiftProcessingException ex)
    when (
        ex.Reason
        ==
        GiftFailureReason.ExternalTimeout)
```

may be reasonable.

But if:

```EXAMPLE / FLOW
Pending

Completed

Cancelled
```

are ordinary business states, prefer normal control flow rather than using exceptions solely to decide among those states.

finally vs catch

`catch` answers:

```EXAMPLE / FLOW
What should I do
with this failure?
```

`finally` answers:

```EXAMPLE / FLOW
What must I do
when leaving this scope,
whether it succeeded or failed?
```

try-finally Without catch

```C#
try
{
    PerformWork();
}
finally
{
    ResetBusyFlag();
}
```

Exception behavior:

```EXAMPLE / FLOW
Exception occurs
    ↓
finally executes
```

```
    ↓
Exception continues upward
```

Nested try-catch

Legal:

```C#
try
{
    try
    {
        PerformWork();
    }
    catch (
        TimeoutException)
    {
        RecoverFromTimeout();
    }
}
catch (
    Exception ex)
{
    LogUnexpected(
        ex);
}
```

But excessive nesting often makes control flow difficult.

Prefer clear architectural handling.

Do Not Wrap Every Line in try-catch

Bad:

```C#
try
{
    Validate();
}
catch
{
}

try
{
    Save();
}
catch
{
}

try
{
    Send();
}
catch
{
}
```

Exception boundaries should reflect:

EXAMPLE / FLOW

```
Recovery boundaries

Resource boundaries

Architectural boundaries
```

not every method call.

Custom Exception Example — Gift Already Finalized

C#

```
public sealed class GiftAlreadyFinalizedException :
    Exception
{
    public GiftAlreadyFinalizedException()
    {
    }

    public GiftAlreadyFinalizedException(
        string message)
        :
        base(
            message)
    {
    }

    public GiftAlreadyFinalizedException(
        string message,
        Exception innerException)
        :
        base(
            message,
            innerException)
    {
    }
}
```

Usage:

C#

```
if (
    Status
    is GiftStatus.Completed
    or GiftStatus.Cancelled)
{
    throw new GiftAlreadyFinalizedException(
        $"Gift {GiftId} has already reached a final state.");
}
```

Whether this should be custom exception or result depends on whether the condition is exceptional in your application workflow.

ASP.NET Core Exception Handling

In a Web API, avoid repeating:

C#

```
try
{
```

```
}
catch
{
    return StatusCode(
        500);
}
```

inside every controller action.

A centralized error-handling boundary is usually cleaner.

Current ASP.NET Core provides exception-handling middleware and the `IExceptionHandler` abstraction for centralized known-exception handling. `AddProblemDetails()` can register the problem-details service used to produce standardized HTTP error bodies.

Conceptual ASP.NET Core Flow

EXAMPLE / FLOW

```
HTTP Request
    ↓
Controller
    ↓
GiftService
    ↓
Repository
    ↓
Exception
    ↑
Not locally recoverable
    ↑
Exception Handling Middleware
    ↓
Log / map
    ↓
ProblemDetails
    ↓
HTTP Response
```

Basic Modern Setup

Conceptually:

C#

```
builder.Services
    .AddProblemDetails();

builder.Services
    .AddExceptionHandler<
        GlobalExceptionHandler>();

var app = builder.Build();

app.UseExceptionHandler();
```

A registered `IExceptionHandler` is invoked by exception-handling middleware; simply registering the handler is not enough—the middleware must participate in the request pipeline.

We'll cover middleware and the ASP.NET Core error pipeline deeply in Topic 38.

IExceptionHandler Example

C#

```
public sealed class GlobalExceptionHandler(
    ILogger<GlobalExceptionHandler> logger)
    :
    IExceptionHandler
{
    public async ValueTask<bool>
        TryHandleAsync(
            HttpContext httpContext,
            Exception exception,
            CancellationToken cancellationToken)
    {
        logger.LogError(
            exception,
            "Unhandled request exception.");

        httpContext.Response.StatusCode = StatusCodes .Status500InternalServerError;

        await httpContext.Response
            .WriteAsJsonAsync(
                new ProblemDetails
                {
                    Status =
                        StatusCodes
                            .Status500InternalServerError,

                    Title =
                        "An unexpected error occurred."
                },
                cancellationToken);

        return true;
    }
}
```

This is only a preview; Topic 38 will cover the pipeline in detail.

Map Known Exceptions

Example conceptual mapping:

EXAMPLE / FLOW

```
ArgumentException
    →
400 Bad Request

GiftNotFoundException
    →
404 Not Found

GiftConflictException
    →
409 Conflict

Unexpected Exception
    →
500 Internal Server Error
```

The mapping belongs at the HTTP boundary, not inside the exception type itself.

Exception Does Not Equal HTTP Status Code

TOPIC 18

A:

```
EXAMPLE / FLOW
GiftNotFoundException
```

does not inherently mean:

```
EXAMPLE / FLOW
404
```

outside HTTP.

The Web API boundary decides:

```
EXAMPLE / FLOW
Application failure
    →
HTTP representation
```

This keeps domain/application code independent from HTTP.

ProblemDetails

ASP.NET Core can produce:

```
EXAMPLE / FLOW
Problem Details
```

responses for HTTP errors.

Typical fields include:

```
EXAMPLE / FLOW
type

title

status

detail

instance
```

Do not expose internal exception details, stack traces, database text, or secrets to production clients. Microsoft explicitly warns that serving sensitive error information is a security risk.

Development vs Production Errors

Development environment may show rich exception diagnostics.

Production responses should generally expose:

```
EXAMPLE / FLOW
Safe public message

Correlation/reference information
```

```
Appropriate HTTP status
```

while detailed exceptions remain in:

EXAMPLE / FLOW

```
Server logs

Telemetry
```

Fundraising Example — Domain Validation

C#

```
public sealed class Gift
{
    public Gift(
        long giftId,
        decimal amount)
    {
        if (giftId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(giftId));
        }

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        GiftId = giftId;

        Amount = amount;
    }

    public long GiftId
    {
        get;
    }

    public decimal Amount
    {
        get;
    }
}
```

These are argument-contract failures.

Fundraising Example — Invalid State

C#

```
public void Refund()
{
    if (
        Status
        !=
        GiftStatus.Completed)
    {
        throw new InvalidOperationException(
            "Only completed gifts can be refunded.");
```

```
    }

    Status = GiftStatus.Refunded;
}
```

The call is inappropriate for current object state.

Fundraising Example — Infrastructure Translation

```C#
public async Task SendReceiptAsync(
    Gift gift,
    CancellationToken cancellationToken)
{
    try
    {
        await _externalEmailClient
            .SendAsync(
                gift,
                cancellationToken);
    }
    catch (
        HttpRequestException ex)
    {
        throw new GiftReceiptDeliveryException(
            $"Receipt delivery failed for Gift {gift.GiftId}.",
            ex);
    }
}
```

Flow:

EXAMPLE / FLOW

```
HTTP failure
    ↓
HttpRequestException
    ↓
Infrastructure/application translation
    ↓
GiftReceiptDeliveryException
```

Fundraising Example — Preserve Cancellation

Do **not** accidentally translate cancellation:

```C#
try
{
    await _externalEmailClient
        .SendAsync(
            gift,
            cancellationToken);
}
catch (
    OperationCanceledException)
    when (
        cancellationToken
            .IsCancellationRequested)
{
    throw;
}
```

```
catch (
    HttpRequestException ex)
{
    throw new GiftReceiptDeliveryException(
        "Receipt delivery failed.",
        ex);
}
```

Cancellation remains cancellation.

Common Mistake — catch Exception Everywhere

Bad:

C#

```
public void Process()
{
    try
    {
        DoEverything();
    }
    catch (
        Exception)
    {
    }
}
```

This hides:

EXAMPLE / FLOW

```
Programming bugs

Infrastructure failures

Cancellation

Unexpected states
```

Common Mistake — Empty catch

C#

```
catch
{
}
```

is usually a severe diagnostic problem unless deliberately justified in a tiny, well-understood scenario.

Common Mistake — throw ex

Bad:

C#

```
catch (
    Exception ex)
{
    throw ex;
}
```

Better:

```C#
catch (
    Exception)
{
    throw;
}
```

The latter preserves the original stack trace.

Common Mistake — Log Then Throw New Exception Without Inner

Bad:

```C#
catch (
    HttpRequestException)
{
    throw new Exception(
        "Failed.");
}
```

Original failure information is lost.

Common Mistake — Catch Just to Rethrow

Pointless:

```C#
try
{
    ProcessGift();
}
catch (
    Exception)
{
    throw;
}
```

If you're not:

EXAMPLE / FLOW

```
Logging

Adding context

Recovering

Translating
```

remove the catch entirely.

Common Mistake — Parsing Exception Messages

Bad:

```C#
catch (
    Exception ex)
```

```
{
    if (
        ex.Message
            .Contains(
                "not found"))
    {
    }
}
```


Use:

```
EXAMPLE / FLOW
Exception type

Error code

Structured property
```

instead whenever possible.

Common Mistake — Exceptions for Every Validation Result

If an API routinely receives invalid form data, throwing and catching an exception for every field can be unnecessary.

Validation systems or result objects may be more appropriate.

Common Mistake — Returning null for Every Failure

Opposite mistake:

```
C#
Gift? GetGift()
{
    try
    {
        ...
    }
    catch
    {
        return null;
    }
}
```

Now:

```
EXAMPLE / FLOW
Not found

Database offline

Timeout

Programming bug
```

all become:

```
EXAMPLE / FLOW
null
```

You have destroyed important failure semantics.

Common Mistake — Retrying Every Exception

Never do:

```C#
catch (
    Exception)
{
    Retry();
}
```

Some failures are:

EXAMPLE / FLOW

```
Permanent

Validation-related

Security-related

Non-idempotent

Programming bugs
```

Retry only errors known to be transient and only when the operation is safe to repeat.

Common Mistake — Logging Sensitive Data

Bad:

```C#
_logger.LogError(
    ex,
    "Payment failed. Card={CardNumber}, Token={Token}",
    cardNumber,
    token);
```

Exceptions/logs can persist for long periods.

Avoid secret/PII leakage.

Common Mistake — Throwing Exception from finally

If:

EXAMPLE / FLOW

```
Original failure
```

is propagating and `finally` throws another exception, the cleanup failure may hide the original diagnostic context.

Avoid throwing from `finally`.

Common Mistake — Catching Cancellation as Error


```C#
catch (
    Exception ex)
{
    _logger.LogError(
        ex,
        "Unexpected failure.");
}
```

also catches:

```EXAMPLE / FLOW
OperationCanceledException
```

unless treated specially.

In cancellation-aware code, distinguish expected cancellation from actual failures.

Common Mistake — Async Without await

```C#
try
{
    DoWorkAsync();
}
catch
{
}
```

does not reliably catch later asynchronous failures.

Use:

```C#
await DoWorkAsync();
```

inside the `try`.

Common Mistake — Exposing Stack Trace to API Clients

Bad production response:

```C#
{
  "error": "SqlException...",
  "stackTrace": "..."
}
```

This may expose:

```EXAMPLE / FLOW
Internal namespaces

Database details

Paths
```

```
Implementation information
*
Security-sensitive context
```


Return safe error contracts.

Performance Considerations

Exceptions are optimized for:

```
EXAMPLE / FLOW
Exceptional paths
```

not frequent branching.

Throwing an exception requires:

```
EXAMPLE / FLOW
Creating exception object

Capturing diagnostic state

Finding handlers

Unwinding stack when necessary
```

That is much more expensive than a normal:

```
C#
if
```

branch.

Try Is Usually Cheap; Throw Is Expensive

The presence of:

```
C#
try
```

does not imply every execution is highly expensive.

The expensive path is generally:

```
EXAMPLE / FLOW
Exception actually thrown
```

Therefore:

```
EXAMPLE / FLOW
Use exceptions for exceptions
```

rather than routine outcomes.

Prefer Try APIs in Hot Routine Paths

Examples:

```C#
int.TryParse()

Dictionary.TryGetValue()
```

avoid paying exception costs for expected invalid/missing values.

Microsoft explicitly recommends `Try*` APIs in scenarios where exception cost or routine failure would otherwise be significant.

Do Not Sacrifice Correctness for Micro-Optimization

If an actual exceptional failure occurs:

```EXAMPLE / FLOW
Database unavailable
```

don't avoid throwing merely because:

```EXAMPLE / FLOW
Exceptions are expensive.
```

Correct semantics come first.

Security Considerations

Exception handling has direct security implications.

Avoid exposing:

```EXAMPLE / FLOW
Stack traces

Connection strings

SQL statements containing sensitive data

Filesystem paths

Tokens

Authentication headers

PII
```

to clients.

ASP.NET Core's documentation specifically warns against serving sensitive exception details to users.

Do Not Trust Exception Messages as Authorization

Bad:

```C#
catch (
    Exception ex)
{
    if (
        ex.Message
        ==
```

```
        "Unauthorized")
    {
        ...
    }
}
```


Authorization decisions belong in trusted authorization infrastructure.

Keep Client and Server Error Details Separate

Server logs:

EXAMPLE / FLOW

```
Full exception object

Stack trace

Correlation ID

Internal context
```

Client response:

EXAMPLE / FLOW

```
Safe title

HTTP status

Public validation/error code

Trace/correlation identifier
```

Practical Exercise 1 — Argument Validation

Write:

C#

```
static void ValidateAmount(
    decimal amount)
{
    if (amount <= 0m)
    {
        throw new ArgumentOutOfRangeException(
            nameof(amount));
    }
}
```

Question:

Why not:

C#

```
throw new Exception();
```

Answer:

EXAMPLE / FLOW

```
ArgumentOutOfRangeException
communicates the exact contract failure.
```

Practical Exercise 2 — Invalid State

```C#
public void Cancel()
{
    if (
        Status
        ==
        GiftStatus.Refunded)
    {
        throw new InvalidOperationException(
            "A refunded gift cannot be cancelled.");
    }

    Status = GiftStatus.Cancelled;
}
```

Question:

Why `InvalidOperationException`?

Because:

EXAMPLE / FLOW

```
The method is not legal
for the object's current state.
```

Practical Exercise 3 — Correct Rethrow

Which is correct?

```C#
catch (
    Exception ex)
{
    throw ex;
}
```

or:

```C#
catch (
    Exception)
{
    throw;
}
```

Answer:

```C#
throw;
```

because it preserves the original stack trace.

Practical Exercise 4 — Exception Filter

```C#
catch (
    OperationCanceledException)
```

```
    when (
        cancellationToken
            .IsCancellationRequested)
{
    Console.WriteLine(
        "Caller cancelled.");
}
```

The filter distinguishes a specific contextual condition.

Practical Exercise 5 — finally

```C#
bool busy = true;

try
{
    ProcessGift();
}
finally
{
    busy = false;
}
```

Even when processing throws, normal exception unwinding still executes the cleanup.

Practical Exercise 6 — TryParse

Bad for routine user input:

```C#
try
{
    int value = int.Parse( input);
}
catch (
    FormatException)
{
}
```

Better:

```C#
if (
    int.TryParse(
        input,
        out int value))
{
}
```

Practical Exercise 7 — Custom Exception

Create:

```C#
public sealed class GiftReceiptDeliveryException :
    Exception
{
    public GiftReceiptDeliveryException()
    {
```

```
    }

    public GiftReceiptDeliveryException(
        string message)
        :
        base(
            message)
    {
    }

    public GiftReceiptDeliveryException(
        string message,
        Exception innerException)
        :
        base(
            message,
            innerException)
    {
    }
}
```

Practical Exercise 8 — Preserve Inner Exception

C#

```
catch (
    HttpRequestException ex)
{
    throw new GiftReceiptDeliveryException(
        "Receipt delivery failed.",
        ex);
}
```

Now:

EXAMPLE / FLOW

```
High-level meaning
+
original low-level cause
```

are both available.

Practical Exercise 9 — Async Handling

C#

```
try
{
    await service.ProcessAsync(
        cancellationToken);
}
catch (
    TimeoutException ex)
{
    Console.WriteLine(
        ex.Message);
}
```

The exception from the asynchronous operation is observed when awaited.

Practical Exercise 10 — Choose Exception or Result

TOPIC 18

Scenario:

```
EXAMPLE / FLOW
Gift ID doesn't exist
and lookups frequently miss.
```

Possible design:

```
C#
Gift? gift = await FindAsync(...);
```

Scenario:

```
EXAMPLE / FLOW
Database connection unexpectedly fails.
```

Exception is appropriate.

Different failures deserve different representations.

Interview Questions

1. What is an exception in C#?

An exception is an object derived from `System.Exception` that represents an abnormal failure during program execution.

It can contain:

```
EXAMPLE / FLOW
Message

Stack trace

Inner exception

Additional diagnostic data
```

2. What does `throw` do?

It raises an exception and interrupts normal execution.

The runtime then searches for a compatible handler.

3. What is a `try` block?

A block containing code for which you want exception handling or guaranteed cleanup behavior.

It is followed by:

```
EXAMPLE / FLOW
catch

finally

or both.
```

4. What does `catch` do?

It handles an exception compatible with the specified exception type and optional filter.

5. What does `finally` do?

It provides code that executes when control leaves the `try` under normal exception-unwinding behavior, regardless of whether the try completed normally or through an exception.

6. Is `finally` absolutely guaranteed to run?

No.

Extreme process termination scenarios such as fail-fast termination can prevent normal unwinding.

For normal managed control flow and exception propagation, `finally` is the cleanup mechanism.

7. Can `try` exist without `catch`?

Yes.

Example:

```C#
try
{
}
finally
{
}
```

is valid.

8. Can `catch` exist without `try`?

No.

A `catch` belongs to a `try`.

9. What order should catch blocks use?

From:

EXAMPLE / FLOW

```
Most specific exception

to

Most general exception.
```

10. Why shouldn't you catch `Exception` everywhere?

Because you may hide:

EXAMPLE / FLOW

```
Programming bugs

Cancellation

Infrastructure failures

Unexpected states
```

Catch only what you can meaningfully handle, except at intentional top-level boundaries.

11. What is exception propagation?

When a method doesn't handle an exception, it moves up the call stack until a compatible handler is found.

12. What is stack unwinding?

The runtime exits stack frames while searching for an exception handler and running applicable cleanup/finally logic.

13. What is a stack trace?

Diagnostic information showing the call sequence leading to the exception.

It is crucial for locating the original failure.

14. What is the difference between `throw;` and `throw ex;`?

Inside a `catch`:

```C#
throw;
```

preserves the original stack trace.

```C#
throw ex;
```

updates/restarts the stack trace from the current location and loses useful original-call information.

15. How do you rethrow outside the original catch while preserving dispatch information?

Use:

```C#
ExceptionDispatchInfo
```

to capture and later rethrow the exception.

16. What is an inner exception?

An exception preserved inside another exception.

Example:

```C#
throw new GiftProcessingException(
    "Gift failed.",
    ex);
```

The original:

```C#
ex
```

becomes `InnerException`.

17. When should you wrap an exception?

When translating a lower-level failure into a meaningful higher-level abstraction while preserving the original cause.

18. Should every layer wrap exceptions?

No.

Only wrap when the layer adds meaningful context or abstraction.

Avoid unnecessary chains such as:

EXAMPLE / FLOW

```
DbException
→ RepositoryException
→ ServiceException
→ ApiException
```

with no added value.

19. When should you create a custom exception?

When no existing .NET exception type accurately expresses the failure and callers benefit from distinguishing it programmatically.

20. What should a custom exception be named?

End its name with:

EXAMPLE / FLOW

```
Exception
```

Example:

EXAMPLE / FLOW

```
GiftProcessingException
```

21. What constructors should a custom exception commonly provide?

Microsoft recommends the common forms:

C#

```
Exception()

Exception(string message)

Exception(
    string message,
    Exception innerException)
```

for user-defined exceptions.

22. What exception should be used for a null argument?

Usually:

C#

```
ArgumentNullException
```

Modern .NET also provides:


```C#
ArgumentNullException.ThrowIfNull(...)
```

23. What exception should be used for an out-of-range argument?

```C#
ArgumentOutOfRangeException
```

24. What exception should be used when an object's current state doesn't permit an operation?

```C#
InvalidOperationException
```

25. Should you manually throw `NullReferenceException`?

Normally no.

Use a contract-specific exception such as:

```C#
ArgumentNullException
```

when appropriate.

26. What is an exception filter?

A `when` condition attached to a catch:

```C#
catch (
    SomeException ex)
    when (
        condition)
```

The handler runs only if both the exception type and condition match.

27. What advantage do exception filters have?

They are evaluated before normal stack unwinding for that candidate handler, preserving more original debugging context when deciding whether to handle the exception.

28. Should exceptions be used for normal control flow?

Usually no.

If a condition is routine and expected, prefer:

EXAMPLE / FLOW

```
if

switch

Try*

Result
```

where appropriate.

29. Why use `TryParse` instead of catching `FormatException` repeatedly?

Invalid input is often an expected condition.

`TryParse` expresses that expected branch without repeatedly throwing exceptions.

30. What happens when an async Task method throws?

The returned task becomes faulted and carries the exception.

When awaited, the exception is observed/rethrown to the awaiting caller.

31. Why might `try { SomeAsync(); } catch { }` fail to catch the error?

Because the asynchronous operation isn't awaited.

The `try` may finish before the task faults.

Use:

```C#
await SomeAsync();
```

inside the `try`.

32. Which cancellation exception should you normally catch?

Generally:

```C#
OperationCanceledException
```

rather than specifically `TaskCanceledException`.

33. Is cancellation the same as failure?

Not necessarily.

Cancellation often represents intentional interruption of work rather than an unexpected fault.

34. Can finally be used with async code?

Yes.

```C#
try
{
    await WorkAsync();
}
finally
{
    Cleanup();
}
```

is valid.

35. Should you throw exceptions from finally?

Avoid it.

A new exception from `finally` can mask the exception already propagating. Microsoft recommends against throwing from `finally`.

TOPIC 18

36. What does a `using` statement have to do with exceptions?

`using` ensures disposal when control leaves its scope, and its cleanup semantics are implemented using `try-finally` style behavior.

Topic 19 covers this deeply.

37. Should every catch log the exception?

No.

Logging at every layer produces duplicate noise.

Choose meaningful logging boundaries.

38. Is `catch (Exception)` always bad?

No.

It is appropriate at true top-level boundaries such as:

EXAMPLE / FLOW

```
ASP.NET Core global handler

Worker/message boundary

Application top-level
```

where the purpose is centralized logging/translation.

39. What is exception translation?

Converting a low-level exception into a higher-level exception that better represents the current abstraction.

Example:

EXAMPLE / FLOW

```
HttpRequestException
    ↓
GiftReceiptDeliveryException
```

while preserving the original as `InnerException`.

40. Should exception messages be parsed for business logic?

Avoid it.

Prefer:

EXAMPLE / FLOW

```
Exception types

Structured properties

Provider error codes
```

Messages are intended primarily for people and may change.

41. Why are exceptions expensive?

Throwing involves work such as:

EXAMPLE / FLOW

```
Creating exception object

Capturing stack information

Finding handlers

Potential stack unwinding
```

so exceptions are inappropriate for frequently expected branches.

42. What is the difference between validation and exception handling?

Validation determines whether input satisfies expected rules.

Exceptions represent abnormal execution failures.

Some programming-contract validations legitimately throw argument exceptions, while user-facing validation often produces structured validation results instead.

43. What is `IExceptionHandler` in ASP.NET Core?

It is a centralized ASP.NET Core abstraction for handling known exceptions at the request-pipeline boundary.

Implementations are used by exception-handling middleware.

44. What is `ProblemDetails`?

A standardized HTTP error response model commonly used by ASP.NET Core APIs.

`AddProblemDetails()` registers ASP.NET Core's problem-details service.

45. Should production APIs return exception stack traces?

No.

Internal exception details can expose sensitive information and should generally remain in trusted logs/telemetry, not client responses.

Best Practices

Use specific exception types.

Prefer:

C#

```
ArgumentNullException

ArgumentOutOfRangeException

InvalidOperationException
```

over generic:

C#

```
Exception
```

when a framework exception accurately communicates the failure.

Use:

```
C#
ArgumentNullException.ThrowIfNull(...)
```

for simple null guards.

Use exceptions for exceptional conditions rather than routine branching.

Use:

```
C#
TryParse

TryGetValue
```

when failure is expected.

Catch only exceptions you can:

```
EXAMPLE / FLOW
Recover from

Translate meaningfully

Handle at a defined boundary
```

Do not add a catch solely because an exception might happen.

Never silently swallow unexpected exceptions.

Use:

```
C#
throw;
```

rather than:

```
C#
throw ex;
```

when rethrowing from the current catch.

Preserve low-level failures as:

```
C#
InnerException
```

when wrapping.

Avoid wrapping at every architectural layer.

Use exception filters when handling depends on exception/context conditions.

Do not put important side effects in filters.

Use:

```
C#
finally
```

for scope-exit cleanup when appropriate.

Prefer:

```C#
using
```

for `IDisposable` resources.

Do not throw from `finally`.

Treat:

```C#
OperationCanceledException
```

separately where cancellation is expected.

Always:

```C#
await
```

tasks whose exceptions you need to observe.

Do not log one exception at every layer.

Log at intentional boundaries with useful structured context.

Never log:

EXAMPLE / FLOW

```
Secrets

Tokens

Passwords

Sensitive payment details
```

inside exception messages or metadata.

Use custom exceptions only when they improve the application's semantic contract.

Name custom exception classes:

EXAMPLE / FLOW

```
SomethingException
```

Maintain valid state when operations fail.

Consider result-based designs for routine business outcomes.

Use centralized ASP.NET Core exception handling instead of repetitive controller-level try/catch blocks.

Return safe HTTP error contracts and keep detailed diagnostics on the server.

Quick Revision

Concept	Meaning
Exception	Object representing abnormal failure
`throw`	Raise exception
`try`	Protected execution block

Concept	Meaning
`catch`	Handle matching exception
`finally`	Scope-exit cleanup
Propagation	Exception moves up call stack
Stack unwinding	Runtime exits frames searching for handler
Stack trace	Call path leading to failure
`throw;`	Rethrow preserving trace
`throw ex;`	Rethrow expression, damages original trace context
`InnerException`	Preserved underlying failure
Exception filter	`catch (...) when (...)`
Custom exception	Application-specific exception type
`Try*` method	Routine failure without exceptions
`OperationCanceledException`	Cancellation signal
`using`	Deterministic resource cleanup
`IExceptionHandler`	ASP.NET Core centralized handler
`ProblemDetails`	Standard HTTP error response model

Complete Fundraising Example

Custom Exception

C#

```
public sealed class GiftReceiptDeliveryException :
    Exception
{
    public GiftReceiptDeliveryException()
    {
    }

    public GiftReceiptDeliveryException(
        string message)
        :
        base(
            message)
    {
    }

    public GiftReceiptDeliveryException(
        string message,
        Exception innerException)
        :
        base(
            message,
            innerException)
    {
    }
}
```

Gift Entity

C#

```
public sealed class Gift
{
    public Gift(
        long giftId,
        decimal amount)
    {
        if (giftId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(giftId));
        }

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        GiftId = giftId;

        Amount = amount;

        Status = GiftStatus.Pending;
    }

    public long GiftId
    {
        get;
    }

    public decimal Amount
    {
        get;
    }

    public GiftStatus Status
    {
        get;
        private set;
    }

    public void Complete()
    {
        if (
            Status
            !=
            GiftStatus.Pending)
        {
            throw new InvalidOperationException(
                "Only a pending gift can be completed.");
        }

        Status = GiftStatus.Completed;
    }

    public void Refund()
    {
        if (
            Status
            !=
```

```
            GiftStatus.Completed)
        {
            throw new InvalidOperationException(
                "Only a completed gift can be refunded.");
        }

        Status = GiftStatus.Refunded;
    }
}
```

Repository Contract

C#

```
public interface IGiftRepository
{
    Task<Gift?> FindAsync(
        long giftId,
        CancellationToken cancellationToken);

    Task SaveAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Receipt Sender Contract

C#

```
public interface IGiftReceiptSender
{
    Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Gift Service

C#

```
public sealed class GiftService
{
    private readonly IGiftRepository
        _repository;

    private readonly IGiftReceiptSender
        _receiptSender;

    public GiftService(
        IGiftRepository repository,
        IGiftReceiptSender receiptSender)
    {
        ArgumentNullException.ThrowIfNull(
            repository);

        ArgumentNullException.ThrowIfNull(
            receiptSender);

        _repository = repository;

        _receiptSender = receiptSender;
    }
```

```
public async Task ProcessAsync(
    long giftId,
    CancellationToken cancellationToken)
{
    if (giftId <= 0)
    {
        throw new ArgumentOutOfRangeException(
            nameof(giftId));
    }

    Gift? gift = await _repository.FindAsync( giftId, cancellationToken);

    if (gift is null)
    {
        throw new KeyNotFoundException(
            $"Gift {giftId} was not found.");
    }

    gift.Complete();

    await _repository.SaveAsync(
        gift,
        cancellationToken);

    try
    {
        await _receiptSender.SendAsync(
            gift,
            cancellationToken);
    }
    catch (
        OperationCanceledException)
        when (
            cancellationToken
                .IsCancellationRequested)
    {
        throw;
    }
    catch (
        HttpRequestException ex)
    {
        throw new GiftReceiptDeliveryException(
            $"Receipt delivery failed for Gift {giftId}.",
            ex);
    }
}
}
```

Flow

EXAMPLE / FLOW

```
ProcessAsync(giftId)
    ↓
Validate GiftId
    ↓
Find Gift
    ↓
Gift found?
    ├─ No
    │    ↓
    │ KeyNotFoundException
    │
```

```
    └─ Yes
       ↓
    Complete()
       ↓
    Valid state?
    ├─ No
    │    ↓
    │ InvalidOperationException
    │
    └─ Yes
         ↓
     SaveAsync
       ↓
     Send Receipt
       ↓
    HTTP failure?
    ├─ No → Success
    └─ Yes
         ↓
    HttpRequestException
         ↓
    Wrap as
    GiftReceiptDeliveryException
```

Global ASP.NET Core Boundary

Conceptually:

C#

```
public sealed class GlobalExceptionHandler(
    ILogger<GlobalExceptionHandler> logger)
    :
    IExceptionHandler
{
    public async ValueTask<bool>
        TryHandleAsync(
            HttpContext context,
            Exception exception,
            CancellationToken cancellationToken)
    {
        logger.LogError(
            exception,
            "Request failed.");

        int statusCode =
            exception switch
            {
                ArgumentException
                    =>
                    StatusCodes
                        .Status400BadRequest,

                KeyNotFoundException
                    =>
                    StatusCodes
                        .Status404NotFound,

                InvalidOperationException
                    =>
                    StatusCodes
                        .Status409Conflict,

                GiftReceiptDeliveryException
```

```
                    =>
                    StatusCodes
                        .Status502BadGateway,

                _
                    =>
                    StatusCodes
                        .Status500InternalServerError
            };

        context.Response.StatusCode = statusCode;

        await context.Response
            .WriteAsJsonAsync(
                new ProblemDetails
                {
                    Status =
                        statusCode,

                    Title =
                        statusCode
                        ==
                        StatusCodes
                            .Status500InternalServerError
                            ?
                            "An unexpected error occurred."
                            :
                            "The request could not be completed."
                },
                cancellationToken);

        return true;
    }
}
```

ASP.NET Core's centralized exception mechanisms are designed for this kind of request-boundary handling; `IExceptionHandler` implementations are coordinated through exception handling middleware.

Complete Exception Decision Flow

EXAMPLE / FLOW

```
Something went wrong
        ↓
Is this a routine expected condition?
```

Yes:

EXAMPLE / FLOW

```
Use:
if / switch / Try* / Result
```

No:

EXAMPLE / FLOW

```
Is there a standard .NET exception
that accurately describes it?
```

Yes:

EXAMPLE / FLOW

```
Throw standard exception
```

No:

EXAMPLE / FLOW

```
Would callers benefit from
a distinct semantic exception type?
```

Yes:

EXAMPLE / FLOW

```
Create custom exception
```

Then:

EXAMPLE / FLOW

```
Can current method recover safely?
```

Yes:

EXAMPLE / FLOW

```
Catch specific exception
    ↓
Recover
```

No:

EXAMPLE / FLOW

```
Let it propagate
```

At a true architectural boundary:

EXAMPLE / FLOW

```
Catch / log / translate
    ↓
Safe external response
```

Rethrow Decision Flow

EXAMPLE / FLOW

```
Caught exception
    ↓
Need to keep propagating
same exception?
```

Yes:

C#

```
throw;
```

Not:

C#

```
throw ex;
```

Need a higher-level semantic failure?

EXAMPLE / FLOW

```
Wrap
    ↓
new HigherLevelException(
    message,
    ex)
```

Need to rethrow later/outside handler?

EXAMPLE / FLOW

```
ExceptionDispatchInfo
```

Cleanup Decision Flow

EXAMPLE / FLOW

```
Need cleanup
when scope exits?
    ↓
Resource implements IDisposable?
```

Yes:

EXAMPLE / FLOW

```
using
```

Otherwise:

EXAMPLE / FLOW

```
try-finally
```

when explicit cleanup logic is appropriate.

Topic 19 will develop this further.

Final End-to-End Mental Model

EXAMPLE / FLOW

```
Normal Execution
    ↓
Unexpected failure
    ↓
throw Exception
    ↓
Runtime searches call stack
    ↓
Matching catch?
```

If yes:

EXAMPLE / FLOW

```
Handle / recover / translate
```

If no:

EXAMPLE / FLOW

```
Propagate upward
```

During exit:

EXAMPLE / FLOW

```
finally
    ↓
Cleanup
```

For rethrow:

EXAMPLE / FLOW

```
throw;
    ↓
Preserve original stack
```

For translation:

EXAMPLE / FLOW

```
Low-level exception
    ↓
Higher-level exception
    +
InnerException
```

For async:

EXAMPLE / FLOW

```
Async operation faults
    ↓
Task becomes faulted
    ↓
await
    ↓
Exception observed
```

For Web API:

EXAMPLE / FLOW

```
Service / Repository exception
    ↓
ASP.NET Core global handler
    ↓
Log internally
    ↓
Map safely
    ↓
ProblemDetails / HTTP status
```

The most important lesson is:

TOPIC 18

Exceptions should represent exceptional failures, not routine branching. Catch only what you can meaningfully handle, preserve the original stack trace with `throw;`, keep original causes through `InnerException` when translating, distinguish cancellation from failure, use `finally` / `using` for cleanup, and centralize unhandled HTTP exception translation at the ASP.NET Core boundary rather than scattering try/catch blocks throughout controllers and services.

For your Fundraising Platform, a strong default model is:

EXAMPLE / FLOW

```
Invalid caller argument
    →
ArgumentException family

Invalid Gift state
    →
InvalidOperationException

Routine missing lookup
    →
null / result where appropriate

Meaningful application/infrastructure failure
    →
specific custom exception if needed

Unknown request failure
    →
central ASP.NET Core handler
```

Next roadmap topic: Topic 19 — Resource Management and `IDisposable`.